

Project 3: Machine Learning for Predicting Trading Signals

Hiren Patel

Technical Report

Notebook link -

http://localhost:8888/notebooks/Stock_Market_Data_Part_3.ipynb

Project goal

The goal of this project was to test whether machine-learning models can predict rule-based trading signals (Buy, Sell, or Hold) from the price features I prepared in Project 2. I first built two technical indicators by hand, used them to create the trading signals, and then trained and compared machine-learning models to try to predict those signals. This was a classification task, and the focus was on doing it honestly and understanding the results, not on making real trading decisions.

1. Technical indicators (MACD and RSI)

I built two indicators manually using pandas, without a ready-made library. The first was MACD (Moving Average Convergence Divergence), which compares a fast 12-day exponential moving average of the closing price with a slow 26-day one, and then compares that to a 9-day signal line. When the MACD line crosses above its signal line it suggests upward momentum (a Buy), and when it crosses below it suggests downward momentum (a Sell). The second was RSI (Relative Strength Index), a 0-to-100 score based on the size of recent gains versus losses over 14 days. An RSI below 30 means the stock is "oversold" (a Buy), and above 70 means "overbought" (a Sell). I calculated both indicators separately for each stock so they used only that stock's own history.

2. Creating the trading signal (the target)

I turned each indicator into its own Buy/Sell/Hold signal, then combined them into one final trading signal, which became my target variable. I set a strict rule: the final signal is only Buy when both MACD and RSI agree on Buy, only Sell when both agree on Sell, and Hold in every other case. This strictness had a big effect: out of about 20.9 million rows, roughly 20.85 million were Hold, and only about 7,000 were Buy and 7,000 were Sell. So the target was extremely imbalanced (about 99.9% Hold). I then converted the labels to numbers (Buy = 0, Hold = 1, Sell = 2) so the models could use them.

3. Preparing the data for machine learning

For my input features I used the ones I created in Project 2 (daily return, monthly return, previous-day close, 5-day and 20-day moving averages, 20-day volatility, high-low range, volume change, relative volume, and two extreme-price flags). I deliberately did NOT use MACD, RSI, or their signals as features, because those were used to create the target. Using them would be "target leakage" - like giving the model the answer sheet. The real question I was testing was whether the price features alone could predict the signals.

Because stock data is time-based, I did not shuffle it. I split it by date: rows before 2015 were the training set (about 16.3 million) and rows from 2015 onward were the test set (about 4.6 million). This reflects how a model would really be used - learning from the past to predict the future. I standardized the features so they were all on the same scale, always fitting the scaler on the training data only and then applying it to the test data, so no future information leaked in. I kept the test set at its full, real size and imbalance so my evaluation stayed honest.

4. The machine-learning models

I compared three models: Logistic Regression (a simple linear model), Random Forest (many decision trees combined), and Support Vector Machine (SVM). For Logistic Regression and Random Forest I trained on the full data and used class weights so the models would pay attention to the rare Buy and Sell classes instead of ignoring them.

SVM was a problem. A standard SVM with an RBF kernel could train on a small sample, but it was far too slow to make predictions on millions of rows, because it must compare every row against thousands of support vectors. I therefore focused my main comparison on Logistic Regression and Random Forest, and only included SVM in smaller sample-based experiments where it could run in a reasonable time. This is a well-known limitation of kernel SVMs on large data.

5. Evaluation results

I evaluated the models using accuracy, and precision, recall and F1-score for each class, plus a confusion matrix. The most important lesson was that accuracy was misleading: a model that predicts Hold for every row scores about 99.9% accuracy while catching zero Buy or Sell signals. So I focused on the Buy and Sell classes.

On the full test set with class weights, Random Forest was the stronger model: about 77% accuracy, and it caught around 60% of Buy signals and 58% of Sell signals. Logistic Regression was weaker: about 50% accuracy, catching only 25% of Buys and 63% of Sells. However, both models had extremely low precision on Buy and Sell - when they predicted a signal, they were almost always wrong, because they raised hundreds of thousands of

false alarms. So Random Forest was the best of the models, but neither was reliable enough to trust, because every gain in catching signals came with a flood of false alarms.

6. Model tuning and imbalance experiments

To improve a model, I tuned the decision threshold of the Logistic Regression so it only predicted Buy or Sell when it was very confident. This raised accuracy to about 99.8% and removed almost all the false alarms, but it went too far - it then missed almost every real signal (Buy recall fell to about 2%). This clearly demonstrated the precision-recall trade-off: I could reduce false alarms only by also catching far fewer signals.

I also ran extra experiments with different levels of balancing. When I trained on a sample that was still mostly Hold, only Random Forest could catch signals. When I trained on a truly balanced sample (roughly equal Buy, Sell and Hold), all models caught many more signals - Random Forest caught over 90% of both Buy and Sell - but precision and accuracy collapsed because the models then over-predicted signals on the real, imbalanced test set. Across every experiment the pattern was the same: the more I balanced the data, the higher the recall but the lower the precision. Random Forest was consistently the best model at catching signals.

7. Key findings and limitations

My main finding is that the price features alone could not reliably predict these rule-based trading signals. The models could be pushed to catch the signals, but never with good precision - there was no setting that gave both high recall and high precision. This happened for three reasons: the target was extremely rare (99.9% Hold), the features did not clearly separate signal days from Hold days, and the most informative indicators (MACD and RSI) had to be excluded to avoid leakage.

The main limitation of the approach is the target itself. Because it required both MACD and RSI to agree, it was extremely strict and rare, which made it very hard to learn. Handling the imbalance (through class weights or balanced sampling) controlled where the models landed on the precision-recall trade-off, but it could not beat the trade-off. In a future project, more predictive features (other technical indicators, longer-term trends, or outside data like news and earnings), a stronger algorithm such as gradient-boosted trees, or a less strict target would likely help.

Finally, these predictions would not guarantee profitable trading. A Buy signal is only a rule, not a real prediction that the price will rise; the models had very low precision, so most of their signals were false alarms; and real trading also involves fees, timing and risk that this project does not model. This was a machine-learning classification exercise, not financial advice.